

Method Identity and the Limits of Formalism

Eric Cafritz*

2026

DOI: 10.5281/zenodo.21678929

Abstract

This essay is an extension of method models described in *The Science of Methods* by Eric Cafritz, DOI 10.5281/zenodo.18733134 (referred to below as the "Register"), and in particular in the companion paper *A Sheaf-Theoretic Model of Methods*, DOI 10.5281/zenodo.20312085 (the "Sheaf-Theoretic Model").

Parts I and II of the Register supply the foundations and grammar for the execution model in this essay. To the extent required, defined terms from the Register and the Sheaf-Theoretic Model are recapitulated below.

Much of the Register, as well as the Sheaf-Theoretic Model, concerns the retrospective examination of established or crystallized methods, whose configurations have been fixed by design or practice. This essay describes methods in application in formal terms. It presents executions as sampled runs, from which patterns in a method can be read, together with the records of completed applications.

This execution model investigates which characteristics carry or preserve method identity, so that a method remains distinctively recognizable despite variation among the steps taken in its applications. The essay also focuses on the limits of formalization, examining the points at which an agent's determination is required in

*Copyright © 2026 Eric Cafritz. All Rights Reserved.

application. At each such point, the ensuing formal consequences depend upon the determination supplied by the agent.

Introduction

Human thought lies at the center of the Science of Methods, and of the execution model.

Methods are human constructs. Some are so formal that they may seem inhuman.

A method is a projection of thought onto a sequence of steps. In the Register's sense, a projection selects a relation-pattern in a complex field and represents it as a reduction that exposes its dependencies. It strips away detail and preserves selected relations. A method preserves the dependency that thought placed among its steps. It drops the rest. The projection preserves the identity of the method. Formalism does not follow the thought left behind.

The selections that constitute a method are agentive acts. They fix its purpose, its relevant relations, and its characteristic steps. These selections form the basis of a method's identity: agency in the making and identity in what the making leaves operative. Formalism can describe the evidence of these selections in the recurrence between applications. It does not fully determine the selecting.

In projection, thought may be translated into or delegated to method. Translation re-interprets thought into individual steps. Steps contain and substitute for thought to the extent necessary for application. Delegation hands the application to another agent, a machine or an algorithm.

In the Register a machine is ideational: a constructed arrangement of relations that, once set in motion, produces intelligible consequences by its internal organization. It takes inputs into a model and yields outputs not individually specified in advance. The toy model, the plausibility machine, and the n-tuple matrix defined in the Register are machines in this sense.

The relationship between thought and method is a machine inter-

face. It transpires through ideational devices: components, techniques, metaphors, worktables, containers, superstructures.

An ideational device is a small, compact machine. A sharp metaphor is a device; an n-tuple matrix is a machine; the difference is one of scale. A component or technique stands at the compact end, an assembly of components at the built end. Metaphors, worktables, containers, and superstructures range across the same axis, from the single sharp figure to the elaborated structure. Thought is projected onto steps through ideational construction, compact or assembled, according to what the method requires.

Some methods may be constructed so as to be automatic in execution. Others require or invite human participation.

The execution model applies in its formal terms to a fully automated method, but its critical, defining junctures are the points at which a human agent would intervene, either of necessity for continuation or by definition. These points mark the boundary of method identity and formalism.

The mixture of prose and mathematics in the Register and in the execution model is analogous to human agency and automation in the description of method. The prose is where the human stands. Though designed at some point by human thought, the formal part runs on its own between the agent's determinations.

Steps and configurations

In method, a step is a deliberate act altering a configuration in response to selected relations. A configuration is an arrangement of the relations that the practitioner has designated, ordered, or set aside for further study. A step produces the configuration from which further steps proceed.

A method includes step-sequences. Their inclusion in a method is fixed at the outset or determined during application of the method. The formal model described below records inclusion by a selector, a two-valued function on step-sequences. The selector assigns 1 to a

step-sequence included in the method and 0 to a step-sequence outside it. The selector's domain is step-sequences; the content of steps and of resulting configurations is distinct from the selector's function.

The application of a method may be represented at coarser or finer grain, as one step, or as several distinct steps. Resolution is the determination of that grain. A method may fix its own resolution by design, as a method of successive refinement does by prescribing its subdivisions; or resolution may be set in the course of an application and vary throughout, as required by the inquiry.

A single step may represent a continuous action. Raising a temperature from one value to another or drawing a curve may be expressed in one step. A continuous result may also be produced over several steps: a continuous curve may be drawn in connected strokes, each a single step; or continuity may be approximated through discrete steps, in the way that Archimedes' inscribed polygons approach the circle.

Steps may be connected or concatenated into a single course, each proceeding from the configuration reached before it, or steps may be built up separately into distinct configurations, as in the configurations on the board of a game of Go.

Steps or configurations may overlap, covering common ground. Some methods require agreement on overlapping parts, as adjacent map sheets must agree along shared margins.

Base case

The execution model considers one method m_U , first at the level of a single application and then at the level of an indexed collection. The purpose of m_U is treated as fixed throughout the applications considered. Applications begin from the common empty initial configuration assigned to the empty step-sequence, defined in the formal basis below. These are simplifying restrictions. Later development may consider method classes and instances, relations among methods, combinations of methods, sub-methods, varying initial configurations, and changes of purpose during application.

The model uses three agentive terms. The originating agent or community is the agent or community through whose inquiry the method is formed. The practitioner applies the method. The studying agent selects methods and applications for modeling and study.

Formation proceeds through attention to a domain under a purpose; designation of relations as objects and constitution of those objects in arrangements; alteration of configurations through deliberate steps; designation of configurations for continuation; and testing and refinement of components and techniques. These are agentive acts.

Formal basis

The execution model adopts the formal objects and notation of the Sheaf-Theoretic Model. Parts I and II of the Register supply their conceptual basis. The following restatement establishes the terminology.

Let U be a domain, that is, an expanse in which elements may exist. A difference in U is any distinction of one of the following kinds: between the presence and absence of an element, between elements, between collections of elements, or between an element and the rest of the domain. A relation in U is a retained difference. An object in U is a relation taken as an element of inquiry. Within an application, the practitioner brings an object into inquiry by designating the relation as distinct.

An arrangement in U is an ordered collection of relations in U . A configuration in U is an arrangement of the relations in U that the practitioner has designated, ordered, or set aside for further study.

A structure in U is a configuration designated for continuation, that is, incorporated into the method as a point of departure for further steps, as such steps are defined above. A configuration acquires the role of structure when further steps proceed from it. Continuation is further progression from such a configuration.

In the Register's vocabulary, designation determines what will count as an object; constitution is the act of placing designated objects in an arrangement for study; naming fixes a designated object so it can be

referred to, compared, and used in arrangements. Each is an agentive act. In the formal construction below, D_{m_U} is the class of configurations with respect to which a structure is designated for continuation, and δ_{m_U} assigns to each such configuration the structure from which continuation proceeds.

Let $\mathcal{R}(U)$ denote the class of relations in U , let $\mathcal{A}(U)$ denote the class of arrangements of relations in U , and let $\mathcal{C}(U)$ denote the class of configurations in U . Let $\mathcal{R}(U)^*$ denote the class of finite sequences on $\mathcal{R}(U)$, including the empty sequence.

Let Σ_U denote the set of available steps after the first retained difference in U . Let Σ_U^* be the free monoid on Σ_U , with identity the empty step-sequence ε_U . Its elements are the finite step-sequences in U , including ε_U .

The configuration map

$$E_U : \Sigma_U^* \longrightarrow \mathcal{C}(U)$$

assigns each step-sequence its resulting configuration.

The use here of the empty step-sequence follows the free-monoid convention adopted by the Sheaf-Theoretic Model. Annex 0 of the Register instead uses a free semigroup with no identity.

The empty step-sequence gives the initial configuration

$$c_0 = E_U(\varepsilon_U),$$

the configuration of the empty arrangement in U before any step is taken.

Define

$$s \equiv_U s' \iff E_U(s) = E_U(s').$$

The configuration quotient is

$$Q(U) = \Sigma_U^* / \equiv_U .$$

Its elements are the $\equiv_U$ -equivalence classes of step-sequences, where

$$[s]_{\equiv_U} = \{s' \in \Sigma_U^* : s' \equiv_U s\} .$$

The quotient-configuration map is

$$\text{Quot}_U : Q(U) \longrightarrow \mathcal{C}(U), \quad \text{Quot}_U([s]_{\equiv_U}) = E_U(s).$$

The definition of $\equiv_U$ gives every member of $[s]_{\equiv_U}$ the common result $E_U(s)$, so Quot_U is well-defined. Each element of $Q(U)$ is a class of step-sequences. Its image under Quot_U is the corresponding quotient-configuration in $\mathcal{C}(U)$. A quotient-configuration is a configuration viewed through the class of step-sequences that determine it.

These objects and operations form the many-sorted algebra

$$\begin{aligned} \mathbf{Rel}(U) = (\Sigma_U^*, \mathcal{R}(U), \mathcal{R}(U)^*, \mathcal{A}(U), \mathcal{C}(U), Q(U); \\ \text{Concat}_U, \text{Arr}_U, \Gamma_U, E_U, \text{Quot}_U), \end{aligned}$$

with typed operations

$$\text{Concat}_U : \Sigma_U^* \times \Sigma_U^* \rightarrow \Sigma_U^*, \quad \text{Arr}_U : \mathcal{R}(U)^* \rightarrow \mathcal{A}(U),$$

$$\begin{aligned} \Gamma_U : \mathcal{A}(U) \rightarrow \mathcal{C}(U), \quad E_U : \Sigma_U^* \rightarrow \mathcal{C}(U), \\ \text{Quot}_U : Q(U) \rightarrow \mathcal{C}(U). \end{aligned}$$

Here Concat_U concatenates step-sequences, with identity ε_U , so $(\Sigma_U^*, \text{Concat}_U)$ is the free monoid on Σ_U ; Arr_U sends a finite sequence of relations to its arrangement; Γ_U sends an arrangement to the configuration it determines; E_U sends a step-sequence to its resulting configuration; and Quot_U sends an equivalence class of step-sequences to the quotient-configuration it determines.

A method on U is

$$m_U = (P_{m_U}, S_{m_U}, C_{m_U}, Q_{m_U}, \Theta_{m_U}),$$

together with a designation (D_{m_U}, δ_{m_U}) for its selected step-sequences. The selector

$$P_{m_U} : \Sigma_U^* \longrightarrow \{0, 1\}$$

determines

$$S_{m_U} = \{s \in \Sigma_U^* : P_{m_U}(s) = 1\}.$$

Purpose establishes the direction of inquiry, in accordance with which an agent selects steps. In the formal representation, the selector P_{m_U} records which step-sequences in Σ_U^* belong to the method, whether their inclusion is fixed in the method's formation or determined during application.

The configurations produced by the selected step-sequences are

$$C_{m_U} = \{E_U(s) : s \in S_{m_U}\} \subseteq \mathcal{C}(U).$$

Let $\mathcal{S}(U) \subseteq \mathcal{C}(U)$ denote the class of structures in U , that is, the configurations in U designated for continuation.

Designation refers to a class $D_{m_U} \subseteq C_{m_U}$ of configurations with respect to which a structure is designated for continuation and a map

$$\delta_{m_U} : D_{m_U} \longrightarrow \mathcal{S}(U)$$

assigning to each such configuration the structure from which continuation proceeds. A common structure may be assigned to distinct configurations in D_{m_U} . The pair (D_{m_U}, δ_{m_U}) is a record of the designating act.

The corresponding quotient-configurations are

$$Q_{m_U} = \{\text{Quot}_U([s]_{\equiv_U}) : s \in S_{m_U} \text{ and } E_U(s) \in D_{m_U}\} \subseteq \mathcal{C}(U).$$

The structure map

$$\Theta_{m_U} : Q_{m_U} \longrightarrow \mathcal{S}(U)$$

is given by

$$\Theta_{m_U}(\text{Quot}_U([s]_{\equiv_U})) = \delta_{m_U}(E_U(s)).$$

P_{m_U} selects the step-sequences belonging to the method, C_{m_U} contains their resulting configurations, Q_{m_U} contains the quotient-configurations corresponding to those resulting configurations with respect to which a structure is designated for continuation, and Θ_{m_U} assigns the structure from which continuation proceeds.

Determinations in application

In the Register, determinations by the agent of plausibility and of coherence govern the taking of steps and the assessment of the configurations reached. These determinations guide step decisions at variable junctures. Obviousness, where it arises as an observed phenomenon in the course of the application, also serves as a signal for continuation and validation.

Two roles of obviousness

In continuation, obviousness marks the limit of plausibility. Plausibility guides the practitioner's determination to take a step while veracity remains unsettled. At the limit signaled by obviousness, the further step presents itself as dictated by local circumstance and can be taken immediately.

In validation, obviousness bears on the configuration reached. A configuration may present itself as inevitable and thereby indicate that

the current orientation remains appropriate to purpose. In some methods, this presentation also discloses the point at which the practitioner deems the purpose of the application to have been met.

Obviousness in an application may show through the steps taken, the configurations reached, the structures designated for continuation, or the practitioner's judgment that purpose has been met. The execution trace records steps and configurations; designation for continuation and any determination of completion belong to the application. Formalization of an explicit test for obviousness would be method-specific.

Plausibility, coherence, and taken steps

Plausibility and coherence describe two determinations in methodic progression. Plausibility bears on the practitioner's determination to take a step while veracity remains unsettled. Coherence bears on the determination that the configuration reached supplies a sufficient point of departure for continuation.

In a crystallized segment of a method, prior inquiry may have settled a reliable step or sequence of steps, and application follows the established progression. At a variable juncture, the practitioner determines the further step in response to the developing configuration. A single method may combine durable stretches with variable junctures, and application may disclose a further step during execution.

The execution traces, defined below, present the resulting histories of steps taken and configurations produced. The representation of a completed application by its finite step-sequences is retrospective. It accommodates sequences settled in advance, sequences discovered during application, and sequences combining both forms of progression.

The formal basis distinguishes certain results of these determinations at the method level. The selector P_{m_U} identifies the step-sequences belonging to m_U , and its selected set is S_{m_U} . The designation (D_{m_U}, δ_{m_U}) specifies, at the method level, the configurations at which continuation is designated and the structure from which it proceeds. The structure map Θ_{m_U} re-expresses δ_{m_U} on the quotient-configurations in Q_{m_U} .

The execution analysis uses the selected set S_{m_U} and examines its bearing on method-identity, deformation, and agency. The designation (D_{m_U}, δ_{m_U}) , the quotient-configurations Q_{m_U} , and the structure map Θ_{m_U} are inherited from the Sheaf-Theoretic Model; the execution constructions use S_{m_U} alone.

Application and execution

An application of m_U is the taking of its steps until its purpose is deemed to have been met, the application is deemed to have failed, revision is deemed to be required, or the taking of steps is terminated before any such determination. An application may include one or more executions.

Fix one application a of m_U , and let $S^a \subseteq S_{m_U}$ be the set of selected step-sequences taken in that application. For each

$$s = \sigma_1 \cdots \sigma_n \in S^a,$$

set $s_{\leq 0} = \varepsilon_U$, and for $0 \leq t \leq n$ define

$$s_{\leq t} = \sigma_1 \cdots \sigma_t, \quad c_t = E_U(s_{\leq t}).$$

In particular $s_{\leq 0} = \varepsilon_U$ and $c_0 = E_U(\varepsilon_U)$ is the initial configuration.

Membership of s in S_{m_U} imposes no prefix-closure condition on S_{m_U} . For $t < n$, a prefix $s_{\leq t}$ may therefore lie outside S_{m_U} , and the corresponding configuration c_t may lie outside C_{m_U} . The empty prefix $s_{\leq 0} = \varepsilon_U$ itself may lie outside S_{m_U} . The execution trace preserves these prefixes and configurations as stages of the application.

The resulting configuration of s is

$$c_n = E_U(s).$$

A determination that the purpose has been met, that the application has failed, or that revision is required may present itself as obvious

or be reached through further assessment. Each is the practitioner's determination.

The terminal status of the application is

$$e \in \{\text{met, failed, revision, terminated}\},$$

the practitioner's determination that purpose was met, the application failed, or revision is required, or else that the taking of steps terminated before any such determination.

For each $s \in S^a$, the execution associated with s is the ordered trace

$$\gamma_s = ((s_{\leq t}, c_t))_{t=0}^n,$$

together with the terminal status e of its application.

Execution state and transition

For the execution trace

$$\gamma_s = ((s_{\leq t}, c_t))_{t=0}^n,$$

define the execution state at time t by

$$x_t = (s_{\leq t}, c_t).$$

The initial state is $x_0 = (\varepsilon_U, c_0)$.

For $0 \leq t < n$, the next step σ_{t+1} gives the transition

$$x_t \xrightarrow{\sigma_{t+1}} x_{t+1},$$

where

$$x_{t+1} = (s_{\leq t}\sigma_{t+1}, E_U(s_{\leq t}\sigma_{t+1})).$$

The sequence $(x_t)_{t=0}^n$ is the labeled state path of the execution trace γ_s . When $n \geq 1$, the first step is the transition $x_0 \xrightarrow{\sigma_1} x_1$ from the initial state. When $n = 0$, the execution takes no step; its state path is the single initial state $x_0 = (\varepsilon_U, c_0)$.

Families of executions

Let $\mathfrak{A}$ be an index set for a collection of applications of m_U selected for study. For each $a \in \mathfrak{A}$, select one step-sequence

$$s^a = \sigma_1^a \cdots \sigma_{n_a}^a \in S^a \subseteq S_{m_U}$$

for inclusion in the execution family. For $0 \leq t \leq n_a$, define

$$s_{\leq t}^a = \sigma_1^a \cdots \sigma_t^a, \quad c_t^a = E_U(s_{\leq t}^a),$$

and let its execution trace be

$$((s_{\leq t}^a, c_t^a))_{t=0}^{n_a}.$$

Since $s_{\leq 0}^a = \varepsilon_U$ for every a , the traces share the initial configuration $c_0^a = E_U(\varepsilon_U) = c_0$. Let e^a be the terminal status of application a , and let the execution be

$$\gamma^a = (((s_{\leq t}^a, c_t^a))_{t=0}^{n_a}, e^a),$$

the trace together with its terminal status.

The corresponding execution family is

$$\Gamma_{\mathfrak{A}}(m_U) = (\gamma^a)_{a \in \mathfrak{A}}.$$

Pairwise comparison of executions

For distinct $a, b \in \mathfrak{A}$, define the execution pair

$$\gamma^{a,b} = (\gamma^a, \gamma^b).$$

The execution pairs form the indexed family

$$\Gamma_{\mathfrak{A}}^{(2)}(m_U) = (\gamma^{a,b})_{(a,b) \in \mathfrak{A} \times \mathfrak{A}, a \neq b'}$$

indexed rather than gathered as a set, so that distinct applications remain distinct even where their traces coincide. Interchanging a and b transposes the derived relations defined below, so ordered and unordered comparison carry the same information.

Each execution pair determines a step-sequence pair, a configuration-path pair, and a terminal-configuration pair:

$$\pi_{\Sigma}(\gamma^{a,b}) = (s^a, s^b),$$

$$\pi_{\mathcal{C}}(\gamma^{a,b}) = \left((c_t^a)_{t=0}^{n_a}, (c_u^b)_{u=0}^{n_b} \right),$$

and

$$\pi_T(\gamma^{a,b}) = (c_{n_a}^a, c_{n_b}^b).$$

At the step-sequence level, the pair permits comparison of the occurrence, multiplicity, and order of steps. At the configuration level, it permits comparison of the paths and their terminal configurations. The studying agent designates which relations the comparison retains.

For the configuration paths, define the coincidence relation

$$M^{a,b} = \left\{ (t, u) \in \{0, 1, \dots, n_a\} \times \{0, 1, \dots, n_b\} : c_t^a = c_u^b \right\}.$$

Equivalently,

$$(t, u) \in M^{a,b} \iff s_{\leq t}^a \equiv_U s_{\leq u}^b.$$

Since $c_0^a = c_0^b = c_0$, the pair $(0, 0)$ lies in $M^{a,b}$.

A coincidence $(t, u) \in M^{a,b}$ is nonterminal if $t < n_a$ and $u < n_b$. At a nonterminal coincidence $(t, u) \in M^{a,b}$, the successor-step pair is

$$\text{Succ}^{a,b}(t, u) = (\sigma_{t+1}^a, \sigma_{u+1}^b).$$

The observed branch set of the execution pair is

$$B^{a,b} = \left\{ (t, u) \in M^{a,b} : t < n_a, u < n_b, \sigma_{t+1}^a \neq \sigma_{u+1}^b \right\}.$$

$B^{a,b}$ records positions at which two applications reach the same configuration and proceed by different steps. When $n_a, n_b \geq 1$, the pair $(0, 0)$ lies in $B^{a,b}$ if and only if the first steps differ, $\sigma_1^a \neq \sigma_1^b$; if either selected step-sequence is empty, no successor step is taken from its initial configuration and $(0, 0) \notin B^{a,b}$. Together, the execution pairs and their derived relations provide a history of recurrence and variation among applications of m_U .

Tolerated variation and retained identity

The executions in the family share the initial configuration $c_0 = E_U(\varepsilon_U)$. For each $a \in \mathfrak{A}$, the configurations are $c_t^a = E_U(s_{\leq t}^a)$ for $0 \leq t \leq n_a$, with $c_0^a = c_0$.

Let $C_{\mathfrak{A}}^{\text{next}}$ be the set of configurations from which a further step is taken in an application $a \in \mathfrak{A}$:

$$C_{\mathfrak{A}}^{\text{next}} = \{c_t^a : a \in \mathfrak{A}, 0 \leq t < n_a\}.$$

For each $c \in C_{\mathfrak{A}}^{\text{next}}$, define the observed next-step set

$$N_{\mathfrak{A}}(c) = \{ \sigma_{t+1}^a : a \in \mathfrak{A}, 0 \leq t < n_a, c_t^a = c \}.$$

An observed branch point is a configuration followed by more than one observed next step:

$$\text{Br}_{\mathfrak{A}}(m_U) = \{ c \in C_{\mathfrak{A}}^{\text{next}} : |N_{\mathfrak{A}}(c)| > 1 \}.$$

The observed tolerance profile of m_U over the execution family is

$$\text{Tol}_{\mathfrak{A}}(m_U) = \{ (c, N_{\mathfrak{A}}(c)) : c \in C_{\mathfrak{A}}^{\text{next}} \}.$$

The coincidence and branch relations of the pairwise comparison are indexed from the initial configuration. For distinct $a, b \in \mathfrak{A}$, the pair $(0, 0)$ lies in $M^{a,b}$; when $n_a, n_b \geq 1$, it lies in $B^{a,b}$ if and only if the first steps differ, $\sigma_1^a \neq \sigma_1^b$.

Proposition 1 (Branches populate the tolerance profile). *For distinct $a, b \in \mathfrak{A}$, if $(t, u) \in B^{a,b}$, then the common configuration $c = c_t^a = c_u^b$ is an observed branch point, $c \in \text{Br}_{\mathfrak{A}}(m_U)$, and $(c, N_{\mathfrak{A}}(c)) \in \text{Tol}_{\mathfrak{A}}(m_U)$ with $|N_{\mathfrak{A}}(c)| > 1$.*

Proof. By the definition of $B^{a,b}$, the pair (t, u) lies in $M^{a,b}$ with $t < n_a$, $u < n_b$, and $\sigma_{t+1}^a \neq \sigma_{u+1}^b$. From $(t, u) \in M^{a,b}$ the configurations coincide, $c_t^a = c_u^b$; write c for this configuration. Since $t < n_a$ and $u < n_b$, applications a and b each take a further step from c , so $\sigma_{t+1}^a \in N_{\mathfrak{A}}(c)$ and $\sigma_{u+1}^b \in N_{\mathfrak{A}}(c)$ by the definition of $N_{\mathfrak{A}}$. These steps differ, so $|N_{\mathfrak{A}}(c)| > 1$, whence $c \in \text{Br}_{\mathfrak{A}}(m_U)$ and $(c, N_{\mathfrak{A}}(c)) \in \text{Tol}_{\mathfrak{A}}(m_U)$. $\square$

In particular, when both selected executions take a first step, the pair $(0, 0)$ lies in $B^{a,b}$ whenever those first steps differ, marking different first steps taken from the common initial configuration c_0 . Since every s^a belongs to S_{m_U} , each member of $N_{\mathfrak{A}}(c)$ occurs within a step-sequence selected by m_U .

The set $N_{\mathfrak{A}}(c)$ aggregates the next steps at every occurrence of c in the family, both as between applications and between positions within one application. A configuration lies in $\text{Br}_{\mathfrak{A}}(m_U)$ when more than one

next step follows it anywhere in the family, whether at occurrences in distinct applications, or at recurrent visits within one application. The pairwise branch sets $B^{a,b}$ record only the cross-application case. Proposition 1 therefore gives one direction; its converse fails, since a branch point may arise within a single application that revisits c .

Because $N_{\mathfrak{A}}(c)$ is a set, it retains the distinct next steps observed at c . Repetitions of steps, their temporal order within an execution, or their association with particular occurrences remain recoverable from the indexed traces, which order the steps within each execution.

The profile gives formal expression to variation tolerated within the observed applications of the method. Here “tolerated” means occurring within step-sequences selected as belonging to m_U . The profile shows, for each $c \in C_{\mathfrak{A}}^{\text{next}}$, the set $N_{\mathfrak{A}}(c)$ of distinct next steps taken from c . Because every execution trace retains its prefix, branches from a shared configuration can also be compared by their preceding step-sequences. This permits examination of whether the observed variation is associated with the configuration alone or with the different routes by which that configuration was reached.

The branch sets and tolerance profiles measure diversity of next steps. Execution traces retain the successor configurations reached after those steps. Two applications may reach the same configuration c , take the same next step σ , and reach different successor configurations, because right-concatenation by σ may take $\equiv_U$ -equivalent prefixes to prefixes in distinct $\equiv_U$ -classes.

Two executions are start-and-result equivalent when they have the same initial and terminal configurations:

$$\gamma^a \sim_{\text{sr}} \gamma^b \iff c_0^a = c_0^b \text{ and } c_{n_a}^a = c_{n_b}^b.$$

Proposition 2 (Start-and-result equivalence through the configuration quotient). *For $a, b \in \mathfrak{A}$, $\gamma^a \sim_{\text{sr}} \gamma^b$ if and only if $c_0^a = c_0^b$ and $s^a \equiv_U s^b$.*

Proof. By definition $\gamma^a \sim_{\text{sr}} \gamma^b$ holds if and only if $c_0^a = c_0^b$ and $c_{n_a}^a = c_{n_b}^b$. The terminal configurations are $c_{n_a}^a = E_U(s^a)$ and $c_{n_b}^b = E_U(s^b)$, so $c_{n_a}^a = c_{n_b}^b$ if and only if $E_U(s^a) = E_U(s^b)$, which is $s^a \equiv_U s^b$ by the

definition of $\equiv_U$. $\square$

Start-and-result equivalence weighs the initial and terminal configurations. The terminal determination gives a finer relation:

$$\gamma^a \sim_{\text{sr},e} \gamma^b \iff \gamma^a \sim_{\text{sr}} \gamma^b \text{ and } e^a = e^b.$$

Both relations are available; the comparison retains whichever the studying agent designates.

It may therefore occur that

$$s^a \neq s^b \quad \text{and} \quad \gamma^a \sim_{\text{sr}} \gamma^b.$$

The two executions then begin from the same configuration and reach the same terminal configuration through different step-sequences; their configuration paths may differ.

In the base case every application shares $c_0 = E_U(\varepsilon_U)$, so the initial-configuration condition holds throughout. It is retained here for later development in which initial configurations vary.

Equivalent results may also be designated at a resolution broader than equality. Let

$$\approx_{\text{res}}$$

be an equivalence relation on terminal configurations selected for the comparison, defined on every terminal configuration $c_{n_a}^a$ for $a \in \mathfrak{A}$. Define

$$\gamma^a \sim_{\text{sr}, \approx_{\text{res}}, e} \gamma^b \iff c_0^a = c_0^b, \quad c_{n_a}^a \approx_{\text{res}} c_{n_b}^b, \quad \text{and} \quad e^a = e^b.$$

This relation induces an equivalence relation on $\mathfrak{A}$:

$$a \sim_{\text{sr}, \approx_{\text{res}}, e}^{\mathfrak{A}} b \iff \gamma^a \sim_{\text{sr}, \approx_{\text{res}}, e} \gamma^b.$$

The quotient

$$\mathfrak{A} / \sim_{\text{sr}, \approx_{\text{res}}, e}^{\mathfrak{A}}$$

groups application indices according to the common initial configuration, equivalent result, and terminal determination of their selected executions. For any $a \in \mathfrak{A}$, call its equivalence class

$$\text{Def}_{\mathfrak{A}, \approx_{\text{res}}, e}(a) = [a]_{\sim_{\text{sr}, \approx_{\text{res}}, e}^{\mathfrak{A}}}$$

the observed deformation class associated with the selected execution γ^a . The executions indexed by this class begin from the same configuration, reach equivalent results, and end under the same terminal determination, while their step-sequences and configuration paths may differ.

For each deformation class

$$K \in \mathfrak{A} / \sim_{\text{sr}, \approx_{\text{res}}, e}^{\mathfrak{A}}$$

define

$$C_K^{\text{next}} = \{c_t^a : a \in K, 0 \leq t < n_a\}.$$

For each $c \in C_K^{\text{next}}$, define

$$N_K(c) = \{\sigma_{t+1}^a : a \in K, 0 \leq t < n_a, c_t^a = c\}.$$

The observed tolerance profile over the executions indexed by K is

$$\text{Tol}_K(m_U) = \{(c, N_K(c)) : c \in C_K^{\text{next}}\}.$$

For each such K , the extent of the observed deformation is described by comparing the step-sequence projections and configuration paths of the executions indexed by K , and by examining the branch sets

$B^{a,b}$ for distinct $a, b \in K$ and the class-level tolerance profile $\text{Tol}_K(m_U)$. Together, these comparisons, branch sets, and the class-level profile identify the differences accommodated among the executions indexed by K .

The deformation classes group the executions selected from the applications indexed by $\mathfrak{A}$ according to start-and-result equivalence and terminal determination, providing an initial formal account of variation among those executions. Comparison with the corresponding classes of another method lies beyond the one-method base case: it requires a common domain or chosen correspondences among steps, configurations, and result-equivalence, and is reserved for later development.

Scope and limits

The execution analysis takes actual applications as its evidential field. Each application contributes the steps taken and the configurations produced. Execution families then support the study of recurrence, branching, tolerance, and deformation.

When two applications pass through the same represented configuration and continue through different steps, the resulting branch establishes that the represented configuration occurs with more than one observed continuation in that family. Possible sources of the difference include agentive determination, distinct retained histories, tacit relations, and additional circumstances relevant to the application. Assignment of the difference to any one source requires a further determination.

Branch sets, tolerance profiles, and deformation classes express patterns within the execution family selected for study. Their construction depends upon the executions selected from the applications under study, the initial configuration $c_0 = E_U(\varepsilon_U)$ fixed by the configuration map, and, where applicable, the chosen equivalence of results. Enlargement of the execution family may refine these objects and disclose further variation. A claim concerning the full range of variation compatible with method-identity therefore calls for corresponding evidence

and explicit criteria of identity.

The configuration relation

$$s \equiv_U t \iff E_U(s) = E_U(t)$$

shows equality of resulting configuration. Execution traces supply histories and configuration paths.

Formal development is drawn from acts of selection, designation, and comparison. These acts supply the relevant relations, the step-sequences belonging to the method, the configurations designated for continuation, the applications selected for study, the standards of comparison, and the criteria of result-equivalence and method-identity. The dependence of the formal consequences upon determinations supplied through such acts situates agency within the theory and delimits the reach of formal conclusions. Where an agent's determination is required, formal development proceeds only after the agent supplies it.

The execution analysis represents the steps taken and configurations reached in an application. An application may end when its purpose is deemed to have been met, when it is deemed to have failed, when revision is deemed to be required, or when the taking of steps terminates before any such determination. The execution is the trace together with its terminal status. Identical ordered traces ending in different ways represent different executions. Equivalence of results, by contrast, concerns the terminal configuration alone.

The configuration map E_U is a function on step-sequences. $s^a = s^b$ yields the same execution trace in every application. The executions coincide, $\gamma^a = \gamma^b$, when the terminal statuses also coincide, $e^a = e^b$.

An execution is an event situated in an external context beyond the method's represented steps and configurations. That context may affect the taking of steps and appear as variation among execution traces, or may affect the significance or evaluation of its result.

The base case is deterministic at the level of execution traces. In it the domain and the initial configuration are fixed. Contingency enters

through the represented step-sequences and through the terminal determination. Variation of the domain or the initial configuration would be topics for later development.

The present model represents methodic progression by finite step-sequences in the free monoid

$$\Sigma_U^*,$$

including the empty step-sequence ε_U . When $\varepsilon_U \in S_{m_U}$, it represents an execution trace that takes no step. A nonempty execution trace is represented by a finite linear ordering of the steps taken.

This representation establishes the base case for prefixes, transitions, execution traces, execution families, tolerance, and deformation. Within each execution trace, the steps have a linear temporal order. Within an execution family, branching appears when a configuration is followed by more than one next step, whether reached by distinct executions or revisited within one execution.

Steps and configurations may themselves express recursive or nested organization at the chosen level of resolution. Methods whose identity-relevant organization calls for these features to appear directly in the execution object motivate richer representations.

Tree-shaped, nested, recursive, and infinite execution objects provide possible extensions of the present model. Each extension calls for corresponding definitions of execution, comparison, tolerance, and deformation, and for an account of how executions behave under the domain-restriction of the sheaf model, together with results specifying how the finite-word theory extends to the enriched setting. The present finite-word model supplies the base case and the point of comparison for those developments.

Coda — Method as artifact and the constitution of identity

A human method is a made and repeatable artifact. Like an art object or a physical tool, it bears relations introduced through its making. An artwork carries selections of material, gesture, and composition. A tool carries selections of purpose, material, shape, balance, and possible use. These selections become constitutive of the object and remain recognizable through its subsequent history.

A method similarly carries the selections of its originating agent or community. Purpose, name, relevant relations, characteristic steps, possible continuations, and expected outcomes give the method its form. Each application realizes that form under different conditions and produces one or more sequences of configurations. The method persists through those applications as the artifact of the selections made within it.

Method-identity appears at several levels. Within one application, an execution remains one continuing execution while its configurations change. Between applications, method-identity persists through differences among their executions. Between methods, differences of purpose, steps, organization, context, provenance, or character distinguish one method from another. These forms of identity resemble the persistence of an artwork through aging and restoration, or of a tool through use, wear, and repair.

The maker's imprint may be individual, collective, or historically layered. It may appear as the unmistakable character of one agent's work, as the accumulated form of a communal practice, or as the constraints embedded by designers in a tool. In each case, identity is carried by selections that remain operative after the originating act.

The designation of acts as steps reflects the Register's triad of designation, constitution, and naming. Acts are designated as steps; ordered step-sequences and their resulting configurations are constituted as execution traces; and the method is fixed under a name for reference and comparison. The name informs which represented step-sequences are selected as belonging to the method, while the selected step-sequences

give operational content to the name.

Many methods bear the name of the agent who devised them. Such names preserve agentive provenance within the method's public identity: they distinguish the method from alternatives, gather varying applications under a common attribution, and make the maker's imprint explicit.

Pairwise comparison of executions can supply evidence of method-identity within the formal construction. Recurring steps, orderings, transitions, branches, and terminal configurations may disclose an operational signature analogous to the recognizable hand of an artist or the characteristic affordances of a tool.

The boundary between formalism and agency stands at the entrance to the formal model. An agent's acts of designation and naming contribute to the constitution of method-identity. Formal recurrence between applications can then confirm evidence of that identity.

The determinations that constitute a method settle in advance to different degrees. A step may be fixed in formation. Membership is recorded by the selector. A continuation is recorded by the designation. The determination of coherence stays with the practitioner in each application. The judgment that a configuration reached is a sufficient point of departure for continuation is made by the practitioner in the course of the application.

A method is a human artifact whose recognizable identity is carried through time by the agentive selections embodied in its purpose, name, characteristic steps, possible transformations, and history of applications.